`.shape`

What: Tells you the **structure** of the array.

Meaning: “How many things along each axis?”

```
arr.shape
```

Example:

```
(2, 3, 4)
```

Mental hook:

> The blueprint of the data.

If shape is wrong, **everything downstream breaks**.

`.size`

What: Total number of elements.

```
arr.size
```

Example:

```
2 * 3 * 4 = 24
```

Mental hook:

> How many atoms exist, not how they’re arranged.

Used for sanity checks and memory reasoning.

`.dtype`

What: Data type of each element.

```
arr.dtype
```

Examples:

- `int32`
- `float64`
- `bool`

Mental hook:

> What kind of box each value lives in.

Controls **precision, memory, speed**.

`.astype()`

What: Creates a **new array** with converted type.

```
arr.astype(np.float32)
```

Mental hook:

> Recasting data into a different mold.

Use cases:

- Save memory
- Prepare for ML models
- Fix mixed-type issues

Never forget: **original array unchanged**.

Mathematical operations on arrays

What: Element-wise operations, vectorized, no loops.

```
arr + 10
arr * 2
arr1 + arr2
```

Mental hook:

> One instruction, applied everywhere.

This is **why NumPy is fast**:

- No Python loops
- CPU-level vectorization

Aggregation functions

What: Reduce many values into **one summary value**.

Common ones:

```
arr.sum()
arr.mean()
arr.max()
arr.min()
arr.std()
```

With axis:

```
arr.mean(axis=1)
```

Mental hook:

> Collapse information along a dimension.

Axis rule:

- `axis=0` → collapse rows
- `axis=1` → collapse columns
- Higher axis → deeper collapse

One-table mental map

Concept	Question it answers
`shape`	How is data structured?
`size`	How much data exists?

Concept	Question it answers
`dtype`	What is data made of?
`astype`	Should data be recast?
math ops	How to transform data fast?
aggregation	How to summarize data?

Core truth (lock this in)

NumPy arrays are **not collections**. They are **structured memory blocks** with rules.

If you reason in:

- shape → correctness
- dtype → performance
- axis → meaning

You will not misuse NumPy.